
DNG Compare
SAME PHOTOGRAPH. DIFFERENT STORIES.
Metadata A
Make NIKON
Model Z 8
DateTimeOriginal 2024:10:14 17:36:21
ISO 64
FocalLength 24.0 mm
ColorSpace Adobe RGB
Rating ★★★★☆
Keywords Sierra, river, sunset
Software NX Studio
GPSLatitude 37.8653 N
GPSLongitude 119.5384 W
DNG
IPTC
EXIF
XMP
TIFF
MakerNotes
Keywords
Ratings
Color Profile
Location
History
XMP
Metadata B
Make NIKON
Model Z 8
DateTimeOriginal 2024:10:14 17:42:03
ISO 64
FocalLength 24.0 mm
ColorSpace sRGB
Rating ★★★★☆
Keywords Sierra, California, river
Software Adobe Camera Raw
GPSLatitude 37.8653 N
GPSLongitude 119.5384 W
IMAGE PIXELS PRESERVED
SAME PIXELS RICHER STORIES
METADATA MAKES CONNECTIONS

DNG Compare

A manual for raw samples, metadata and exact recovery

Alexy Khrabrov

2026-11-02

Version 1.1.0-1a86e3c2
Source commit 1a86e3c22594dfe9e8b6b33ef5cfbaf19a5a988f

First Pair Press
firstpair.press

Contents

dng-compare: a manual for comparing and preserving DNG archives . . . 2
Choose the question before the tool . . . 3
Install and build . . . 3
A reproducible synthetic example . . . 4
Python comparison command . . . 5
How the Python image comparison works . . . 6
Metadata extraction and categories . . . 7
Interpret the Python reports . . . 8
Resume and protect a comparison . . . 8
Rust bounded scanner . . . 9
Rust metadata reporter . . . 9
Archive embedded XMP bytes . . . 10
Preserve exact original files . . . 10
Native byte workers . . . 11
Full metadata snapshots and queries . . . 11
Develop settings across editors . . . 12
Rust and OCaml extraction pools . . . 14
Year-scoped preservation workflow . . . 16
Finalization and image categorization . . . 17
SQLite and optional Turso . . . 17
Measure workers without disturbing a real job . . . 18
Recovery after a failure . . . 18
Boundaries of this release . . . 19

dng-compare: a manual for comparing and preserving DNG archives

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and the code with him.

A DNG can change without its sensor samples changing. Lightroom or another editor may write new XMP, update a preview, change a rating, or reorganize TIFF metadata. Whole-file checksums then differ, even when the photograph's stored raw samples remain identical. Conversely, a matching filename, file size or stored digest is not enough to establish that two copies contain the same image.

This project separates those questions. It reads actual image blocks, decodes when the Python comparison needs to resolve different encodings, reports metadata differences, and can preserve an exact older file version as a complete original or a verified delta. Python coordinates the work. Rust supplies a bounded no-decoding scan and native workers. OCaml supplies alternative native metadata and byte workers with the same orchestration contract.

The public source is firstpair/dng-compare. The README gives the overview and the HOWTO gives a short path through a comparison. This manual describes what each verdict proves, what it leaves open, and how to reproduce the workflow without a personal photo library.

Choose the question before the tool

Identity	What is compared	Appropriate use
Whole-file identity	Every byte, including XMP, previews, padding and private data	Exact restoration, backup identity and a cloud original's whole-file checksum
Encoded image identity	Ordered image blocks and the selected decoding parameters	Confirm equal stored samples without decompression
Decoded sample identity	Complete arrays of stored sample values after decoding	Resolve different compression, strip layouts or byte order
Metadata equality	Typed fields exposed by ExifTool, under the stated exclusions	Explain settings, timestamps, camera data and layout differences
Rendered image equality	A processing pipeline's output after crop, profiles and edits	Not established by this project
Lightroom native fingerprint	Adobe's native original-digest calculation	Not generated or registered by this project

The distinction matters before any deletion. A source whose raw samples are represented elsewhere may still contain a unique develop history, embedded preview, original-file payload or private metadata. The preservation layer is there to retain an exact recoverable version, rather than equating raw equality with permission to discard its wrapper.

DNG extends TIFF and can carry raw data together with interpretation and other metadata. Adobe publishes the format specification and SDK. The comparator's selected tags and page classification are its implementation scope, not a claim of complete DNG specification conformance. Adobe's DNG resources.

Install and build

The Python comparison needs Python 3.12 or newer, ExifTool, NumPy, tifffile and imagecodecs. The project pins its dependency set through uv. Native Rust targets Unix and is pinned to Rust **1.99.0, edition 2024**. The metadata pool is written in ordinary OCaml. Its historical executable name ends in `-oxcaml` so existing archives and backend selection remain compatible; that name does not require an OxCaml compiler. The OCaml byte worker's C SHA binding uses macOS CommonCrypto.

On macOS, install uv and rustup using their official installation instructions, then install the remaining tools and build:

```
brew install exiftool xdelta ocaml
git clone https://github.com/firstpair/dng-compare.git
cd dng-compare
make setup
rustup toolchain install 1.99.0
make build
make test
```

Run these from the real repository root. `make setup` uses `uv sync`; ordinary Python commands use `uv run python`. Do not substitute a different Rust release without recording and testing that change. Confirm the compilers you actually use:

```
uv run python --version
rustc --version
ocamlopt -version
exiftool -ver
xdelta3 -V
```

`make build-rust` builds the ordinary Rust binaries. `make build-oxcaml` builds `metadata-extract-oxcaml` and `byte-verify-oxcaml`. The Makefile defaults to `OCAMLOPT=ocamlopt` and accepts an explicit compiler or flags override:

```
make build-oxcaml OCAMLOPT=/path/to/ocamlopt OCAMLFLAGS=-O3
```

The older `OXCAMLOPT` override remains available for an existing switch. Linux can use the Python comparator and Unix Rust tools when their dependencies are available; the macOS CommonCrypto OCaml byte binding needs a platform-specific replacement there. Native Windows support is not supplied by these Unix file, process and lock APIs.

The tests create temporary synthetic files. They check comparison, cache invalidation, preservation, bounded parallel work and checkpoint behavior. A passing fixture suite is evidence for those cases, not validation of every camera, codec or DNG layout. Optional Turso probes are separate from the normal build and are discussed below. The publication validation record names the observed compilers, fixture results and remaining untested scopes.

A reproducible synthetic example

This example creates two small DNG-like TIFF files in a dedicated directory. They contain the same 16-bit grayscale samples but different ImageDescription metadata. They are test fixtures rather than certified camera originals.

Start in the repository root, with a new empty `demo` directory. The script refuses to reuse an existing directory:

```
uv run python - <<'PY'
from pathlib import Path
import numpy as np
import tifffile

root = Path('demo')
root.mkdir(exist_ok=False)
(root / 'left').mkdir()
(root / 'right').mkdir()
pixels = np.arange(256, dtype=np.uint16).reshape(16, 16)
for side, description in [('left', 'Older metadata'),
                          ('right', 'Newer metadata with a longer description')]:
    tifffile.imwrite(root / side / 'same.dng', pixels,
                     description=description, metadata=None,
                     extratags=[(50706, 'B', 4, (1, 4, 0, 0), False)])
PY
uv run python compare_dng.py demo/left demo/right --output demo/report
```

Expected results are complete raw coverage, equal auxiliary coverage for this simple fixture, and an embedded ImageDescription difference. The image method should be `encoded_blocks`: different metadata placement has not required sample decoding. Read the actual report rather than relying on this description:

```
uv run python - <<'PY'
import json
from pathlib import Path
s = json.loads(Path('demo/report/summary.json').read_text())
p = json.loads(Path('demo/report/pairs.jsonl').read_text().splitlines()[0])
print(s['complete'], s['left_subsumed_by_right'])
print(p['raw'])
print(p['differences']['embedded'])
PY
```

Run the same comparison again. It should reuse the successful pair if both file stamps are unchanged. Run the Rust scanner against the same generated roots:

```
./rust-scanner/target/release/dng-subset \
  demo/left demo/right demo/rust-report --rate-mib 32
```

To exercise decoded comparison, create another new fixture using the same array but `compression='deflate'` and a different `rowsperstrip` in one copy. The Python comparator should decode and compare the complete arrays. Rust may leave that pair unverified because it does not have a decode fallback. The test suite includes that case and a deliberately changed sample and truncated file.

This is also a suitable input for a preservation rehearsal. Store a delta and restore the older bytes into a new destination:

```
uv run python preserve_apo.py scan demo/left demo/right demo/archive \
  --delta --workers 1 --rate-mib 32
uv run python preserve_apo.py restore demo/archive same.dng demo/restored.dng
./rust-scanner/target/release/byte-verify compare \
  demo/left/same.dng demo/restored.dng
```

The last line should print equal source and restored SHA-256 values followed by `1`. Keep the reference fixture: the delta depends on its exact bytes. Use `--full-copy` in a separate archive if you want a self-contained original instead.

Python comparison command

```
uv run python compare_dng.py LEFT RIGHT [options]
```

Option	Default	Purpose
`--output DIRECTORY`	`dng-report`	Reports and comparison checkpoint
`--workers N`	2	Concurrent image pairs; must be positive
`--batch-size N`	32	Candidate pairs per metadata/checkpoint batch
`--sample N`	Full candidate set	Evenly spaced candidate pairs
`--limit N`	Full candidate set	First N candidate pairs
`--fresh`	Reuse eligible cache	Ignore saved comparison results
`--pair LEFT RIGHT`	Name matching	Compare LEFT with RIGHT; repeatable
`--pairs FILE`	Name matching	Read `LEFT<TAB>RIGHT` pairs, one per line

`--sample` and `--limit` are mutually exclusive. Their unit is a **candidate pair**, not necessarily a unique source image. If one basename has several copies on each side, sampling may select multiple candidates for the same left file and leave other left files untouched.

The program walks both roots recursively, includes extension-case variants of `.dng`, and ignores macOS `._` companion files. Basename matching itself is case-sensitive. Each left physical path is a coverage obligation. For a basename appearing several times, all candidates are scheduled, with the same-relative-path candidate first. Additional right files are ignored. Unreadable directories fail rather than silently disappearing from the scope.

This is a one-way containment check, not a two-way tree diff. To check the reverse direction, use a separate report directory and reverse the roots. A renamed DNG is missing under this matcher even if its bytes exist elsewhere. Use an explicit mapping or an independent identity investigation before treating that as actual original loss.

An explicit mapping replaces name matching for each listed left file; unlisted left files are still matched by name. Give pairs with `--pair LEFT RIGHT` (repeatable) or in a `--pairs` file of `LEFT<TAB>RIGHT` lines, where blank lines and lines starting with `#` are ignored. Each member is an absolute path, a path relative to its own root, or a basename that occurs exactly once under that root. It must be a DNG the inventory found under its root; anything else, or an ambiguous basename, is an argument error (exit 2). Several right files may be listed for one left file, and each is a candidate. `summary.json` records the number of listed pairs as `explicit_pairs`, and `left_only` covers the left files that have no pair of either kind. Root membership and filename matches are candidate selection, never the final image proof.

How the Python image comparison works

The comparator requires a DNGVersion tag in each first page and traverses main and subordinate TIFF pages. It classifies primary raw pages using subfile type zero and its supported photometric codes; other pages go into the auxiliary group. It compares corresponding pages in their traversal order. Different page counts remain unverified instead of being guessed into correspondence.

For each page it validates the declared strip/tile ranges against the physical file length and reads every payload block. SHA-256 includes each block length before its bytes, so block boundaries are part of the encoded comparison. A matching payload digest is accepted on the fast path only with the selected encoding signature, including byte order and the implemented interpretation parameters.

If that path does not match, `tifffile` and `imagecodecs` decode the two pages. Shapes and complete arrays must compare equal; NaNs compare equal for this purpose. A changed decoded array is `different`. A missing decoder, malformed page or decode error is `unverified`, with its actual error retained. Differing compressed bytes alone are not classified as a pixel difference.

Here, raw samples mean the stored sensor or linear samples, including the sample margins, before rendering. Crop, white balance, orientation, DNG opcodes, profiles and develop instructions are not applied. Equal samples are therefore compatible with different appearance in Lightroom. The comparison is not an Adobe SDK validation, a demosaicing test, or proof that an identical damaged pair is healthy. `OriginalRawFileData`, which can contain an embedded original source file, is excluded from image payload and ExifTool comparisons; use whole-file preservation when that content must be retained.

Metadata extraction and categories

The comparison invokes ExifTool once per pending batch, overlapping extraction with image work. It requests JSON, grouped/instance-qualified names, numeric output, structured values, unknown tags and binary contents. Preview/raw image payloads are excluded because the image comparator reads those directly. The portable snapshot route additionally retains the wider grouped output described under metadata archives.

ExifTool's JSON and grouping options are important: structured XMP and binary values can be retained instead of a flattened presentation or a byte-count placeholder. The tool's numeric output is its own representation of metadata; Python does not recover the original TIFF storage type from that JSON. ExifTool's application documentation.

The main comparator normalizes global `CopyN` group numbering so adding one XMP field does not make unchanged EXIF duplicate instances appear changed. Real duplicate values within a group remain retained. It flattens dictionaries for field comparison and labels differences directionally:

- `added`: present on the right and absent on the left.
- `removed`: present on the left and absent on the right.
- `changed`: present on both sides with unequal values.

It separates `embedded`, `filesystem` and `layout`. Directory, filename, access time, ExifTool's own fields and derived Composite fields are excluded from this comparison. File size, modification/change time and permissions can appear as filesystem differences. TIFF image offsets/counts and preview placement belong to layout. A layout difference need not change samples.

Metadata comparison covers what ExifTool exposes under the chosen flags. It does not compare all arbitrary padding, unused bytes or uninterpreted private structures. Warnings and errors are retained. In the compact Python report, large differing values have a canonical JSON length, SHA-256 and prefix instead of the full printed value. The portable gzip archive retains full extracted values for later queries or restoration decisions.

Interpret the Python reports

`summary.json` is updated after each completed batch. Its final `complete` flag means all candidate pairs in the full inventory were processed. It can be true while some left files are missing or different. Missing filenames have no candidate pairs, so an empty candidate workload is not proof of coverage of a nonempty left root.

`left_files_covered` counts unique left paths, separately for raw, all-images and metadata preservation. `left_subsumed_by_right` compares each count with the entire left inventory:

Field	Acceptance
`raw`	At least one right candidate has equal raw pages
`all_images`	A matching candidate has equal raw and auxiliary pages
`metadata_preserved`	A raw-equal candidate has successful metadata extraction, no warnings, and no removed or changed embedded fields

Added embedded metadata is allowed by `metadata_preserved`. That field does not include arbitrary uninterpreted bytes or guarantee that the auxiliary images are equal: these are independent conditions. The qualifying candidate may differ between coverage categories. Consult the individual pair records to identify a specific copy satisfying all the conditions you need.

`counts` reports pairs and verdicts, not unique image coverage. A duplicate-name workload can have both `raw_different` and `raw_equal` pairs while the left file is fully covered by one equal candidate. `metadata_patterns` groups recurring sets of changes; `metadata_tags` includes frequencies and at most three examples per changed field. `verified_payload_bytes` is aggregated from pair results, including reused results; it is not necessarily physical I/O in this invocation.

`pairs.jsonl` is the detailed evidence. `unmatched.json` lists missing left names and all duplicate-name candidate sets. At normal completion, `not-confirmed.json` lists left paths without a confirmed raw copy, including untested paths from a sample. It can be absent or stale after an interruption, so use the current summary and pair/cache evidence in that case.

Exit status is 0 for complete left raw coverage, 1 for unconfirmed coverage, 2 for argument errors, and 130 for a caught keyboard interruption. Metadata changes alone do not fail the raw verdict. Read failures and source changes remain errors, rather than equal results.

Resume and protect a comparison

The Python checkpoint key binds cache version and both absolute paths. Reuse requires matching device, inode, size, nanosecond mtime and ctime on both sources. Successful metadata/image results can be cached; errors and unverified raw or auxiliary groups are retried. Rerunning rebuilds the JSON reports from the current inventory and reusable checkpoint.

A filesystem identity check avoids rereading unchanged files, but it is not a cryptographic freshness guarantee against a privileged actor who can replace bytes while preserving all recorded facts. If the audit requires fresh payload reads, use `--fresh` or a new report directory. A remount can invalidate the Python device stamp and cause a reread.

The Python comparator holds a nonblocking Unix lock for its output directory. Do not edit source metadata, replace images, alter the report database or start a second owner while it runs. Read-only access can still

update filesystem access times; that is excluded from verdicts. Keep output directories outside the photo roots to avoid accidentally treating reports as source material in later archive commands.

Rust bounded scanner

```
dng-subset LEFT RIGHT REPORT [--rate-mib N] [--max-new N]
           [--import-cache FILE] [--after SUMMARY]
```

The binary is `rust-scanner/target/release/dng-subset`. It has one process and checks one pair at a time using 1 MiB buffers. It recognizes classic TIFF and BigTIFF headers, traverses implemented IFD/subIFD paths and confirms equal selected raw blocks plus their encoding tags. It uses direct byte comparisons, not stored DNG digests. The parser has finite limits for IFD counts, tag counts, selected tag sizes and integer arrays. Unsupported or damaged structures remain unverified unless its exact whole-file fallback proves that the bytes are preserved.

The default `--rate-mib 32` accounts for combined raw-data reads in MiB/s; choose any finite number above zero through 512. It is an average pacing budget, not a kernel I/O ceiling for all metadata, filesystem and decoder traffic. `--max-new N` limits newly checked left files, leaving others pending. Matching names and relative-path priority follow the same basic one-way model as Python.

Unlike Python, Rust does not decode different compression/layouts or compare auxiliary previews and metadata. `not-confirmed.tsv` reports `MISSING`, `UNVERIFIED` and `PENDING`. A differing encoded block is unverified, not a proven sample difference. Exit 0 confirms complete raw subsumption; exit 1 leaves coverage unconfirmed; parsing/argument/I/O failures are nonzero as well. Always inspect `summary.json`, which retains `total_left`, `verified_equal`, `missing_name`, `unverified`, `complete` and `left_subsumed_by_right`.

`verified.tsv` is append-only. Its four columns are hex-encoded left path, hex-encoded right path and comma-separated stamps for both files. Each stamp has device, inode, size, mtime and ctime. Resume retains path membership and checks inode, size, mtime and ctime, deliberately ignoring device number across external volume remounts. It does not hash accepted payloads again. There is no cache version/executable hash bound to this TSV format and no automatic report lock; keep the producing source version and one exclusive owner yourself. A new report directory gives a fresh scan.

`--import-cache` trusts already verified records in the same format after current identity checks. It is not a way to admit arbitrary filename matches. The `export_verified.py` helper exports eligible Python raw-equal cache results for explicit roots. Keep its input report and tool provenance.

`--after SUMMARY` waits until the named file contains the literal `"complete": true` and both input directories exist, checking once per minute. It does not parse and require a successful subsumption verdict. A completed prior scan can have missing or unverified files. Use this only as a scheduling aid, not a safety dependency, and do not share report directories across jobs.

Rust metadata reporter

```
dng-metadata IMPORTED_VERIFIED.tsv RUST_VERIFIED.tsv REPORT_DIR BATCH_SIZE
```

This ExifTool reporter reads verified pair ledgers, deduplicates by left path and writes atomic per-batch JSONL files plus `summary.json` and a combined `pairs.jsonl`. `BATCH_SIZE` is 1–64. If there is no imported ledger, supply an existing empty file:

```
mkdir -p reports
touch reports/empty-import.tsv
```

```
./rust-scanner/target/release/dng-metadata \
  reports/empty-import.tsv reports/rust/verified.tsv reports/rust-metadata 32
```

It compares values in full but shows large values with a length and prefix; its compact representation does not include the Python compact-value SHA. It has its own report schema, with `changes` and `status`, rather than Python's `differences` and `metadata_status` fields.

The reporter does not reverify payload equality or current source identity. Existing batches are reused using their index and record count, with no complete manifest/content guard. Keep the pair ledgers, batch size and sources unchanged; use a new output directory after any change. A completed metadata summary can include errors. This reporter is useful for stable, already qualified pairs; the full portable archive is the stronger provenance route.

Archive embedded XMP bytes

```
./rust-scanner/target/release/dng-subset --archive-xmp \
  /media/source/foto archives/source-xmp
```

This extracts TIFF XMP tag 700 bytes under `packets/`, preserving the relative source layout. It records packet size, SHA-256, extraction method and source identity in manifests. On a parser failure it can use ExifTool extraction as a fallback, recorded separately. It does not modify a source DNG.

Per-batch manifests permit resume after checking source stamps and stored packet hashes. The final summary counts files with/without XMP, packets and errors and reports `all_xmp_saved`. A file with no packet is different from a failed extraction. Existing sidecars are outside this command; preserve them separately. Embedded source XMP is not automatically a current cloud develop packet, and a packet hash must never be renamed to impersonate a different requested hash.

Preserve exact original files

The historical filename `preserve_apo.py` now accepts explicit generic roots:

```
uv run python preserve_apo.py scan SOURCE REFERENCE ARCHIVE
    [--project DIRECTORY] [--workers N] [--rate-mib N]
    [--full-copy] [--delta] [--pairs FILE]
uv run python preserve_apo.py restore ARCHIVE RELATIVE_PATH OUTPUT
    [--reference-root DIRECTORY]
```

The scan inventories **all regular files**, not just DNGs, and rejects unsupported file entries such as symlinks. It uses any supplied known-pair ledger and otherwise looks for the same relative path under the reference. An absent reference is preserved as a complete original. Keep source/reference roots stable, writable output separate, and one archive owner at a time.

Mode	Equal whole-file bytes	Different bytes with reference	Missing reference
Default	SHA-bound reference	Complete original	Complete original
`--delta`	SHA-bound reference	Verified VCDIFF delta	Complete original
`--full-copy`	Complete original	Complete original	Complete original

`--full-copy` takes precedence over delta storage. The scan compares complete source/reference bytes, records their SHA-256, validates before/after identities, and rereads saved complete originals. For a delta, xdelta3 encodes the source version against the reference. The archive then decodes it and requires the

reconstructed SHA to equal the observed source SHA. VCDIFF is a standard delta format, rather than an image-specific edit representation. RFC 3284.

`--pairs FILE` preserves only the listed files instead of the whole source. Each JSON line is `{"source": PATH, "reference": PATH}`; the reference may sit under another relative path, and both must lie under their roots. Each run adds to the same archive. Semantic Unison's `unison -semdiff -preservefull` writes these lists for files whose image is identical but whose metadata differs, and preserves them with `dng-archive preserve` (`ocaml-archive/`), the OCaml port of this `--pairs` mode, which writes the same records and deltas: either tool restores what the other preserved.

The scan records source attributes, including mode, times and extended attributes where supported. A restore verifies payload bytes before a no-overwrite commit and reapplies saved xattrs, mode and times. It does not recreate every possible filesystem property: saved uid/gid/flags are not all automatically restored. Cross-platform xattr support also differs. Preserve that distinction from an exact file-payload guarantee.

The public entry points reject an archive output that overlaps a source or reference root. This protects against writing recovery files into the trees being audited. It does not supply an automatic cross-process lock to every low-level API. Use one archive owner and separate output directories.

`summary.json` distinguishes processed completion, errors and `byte_preservation_complete`. `recovery.sqlite3` is the checkpoint, `manifest.jsonl` is the portable record view, `inventory.json` covers source files, and `directories.json` retains directory attributes. Each recovery record says whether it uses `original`, `delta` or a reference alone. The scan never sets `safe_to_delete_source` true by itself.

Default and delta archives depend on retained exact reference bytes. An XMP write can break that dependency despite unchanged samples. Keep another independent reference copy, archive the reference too, or choose full-copy storage. Restore can accept a relocated reference root through `--reference-root`; the relative reference path must be safe and the bytes must still match the recorded hash. An existing output is rejected, so restore into a new location.

Native byte workers

The Python archive layer can select a native worker with `DNG_BYTE_BACKEND`:

```
DNG_BYTE_BACKEND="$PWD/rust-scanner/target/release/byte-verify" \
uv run python preserve_apo.py scan /media/source/foto /media/reference/foto \
  archives/source --delta --workers 2 --rate-mib 64
```

Use `$PWD/byte-verify-oxcaml` to select the OCaml worker on macOS. Both support:

```
hash FILE
compare SOURCE REFERENCE
delta REFERENCE SOURCE DELTA EXPECTED_SOURCE_SHA256
```

`compare` prints source SHA-256, reference SHA-256 and equality `1` or `0`, tab separated. An empty reference argument requests source hashing without a reference. `delta` outputs its own SHA only after reconstruction passes. Workers own bounded buffers and the external xdelta process; Python owns file identity, inventory, checkpoint and acceptance. Direct worker calls do not supply those wider guards. Native byte operations are whole-file operations, not stored-sample or Lightroom-native fingerprint calculations.

Full metadata snapshots and queries

```
uv run python portable_metadata.py SOURCE REFERENCE ARCHIVE
    [--project DIRECTORY] [--batch-size N]
```

The portable snapshot route retains unabridged ExifTool JSON in atomic `metadata/batches/*.jsonl.gz`, independently reads it back, and records batch checksums. It inventories files and before/after identities. `.DS_Store` has an explicit ignored-system classification instead of a metadata failure. Missing references remain visible and do not produce a fictitious comparison.

The original archive schema uses names `apo` for **source** and `bumer` for **reference**. Those field names are retained for compatibility; they do not select a machine or drive. The directional change labels mean:

Historical label	Generic meaning
`added_in_bumer`	Added in the reference
`missing_in_bumer`	Missing from the reference
`changed_in_bumer`	Source and reference extracted values differ

The snapshot keeps wider `-a -G0:1:4 -n -struct -b -u -U` ExifTool output and rejects duplicate JSON keys. It retains arrays, structures and binary metadata values, excluding image payloads and embedded OriginalRawFileData. Its filesystem classification and field names differ from the compact comparator; do not join the two schemas by assuming identical keys.

`metadata_index.py ARCHIVE` creates/rebuilds the searchable `metadata/differences.sqlite3` index from gzip batches. It deduplicates full JSON values by SHA and stores compressed complete values, with coverage and change views. Query only a completed checkpoint after its writer closes:

```
uv run python metadata_index.py archives/source
uv run python - <<'PY'
import sqlite3
from pathlib import Path
p = Path('archives/source/metadata/differences.sqlite3').resolve()
with sqlite3.connect(f'file:{p}?mode=ro', uri=True) as db:
    for row in db.execute('SELECT tag, kind, affected_files FROM field_changes '
                          'ORDER BY affected_files DESC LIMIT 20'):
        print(row)
PY
```

Metadata snapshots are descriptive, not a merge or write-back plan. An added field can be harmless, a timestamp can be bookkeeping, and a rating or crop can be a meaningful user choice. Inspect exact values and decide what to retain.

Develop settings across editors

The metadata snapshots above compare two copies of the same files. A different question is how several editors set up the same photographs, for example after running Auto in Lightroom, Lightroom Classic and Capture One. `develop_settings.py` stores each editor's settings as a separate, write-once **capture** in one archive:

Source (`app`)	Reads	Raw record kept
`xmp`	Settings embedded in each DNG's XMP, or in exported JPEGs with `--extensions jpg` (or `dng,jpg`), via ExifTool; or an existing `compare_dng.metadata` JSONL snapshot (`--snapshot`)	All ExifTool tags
`lrcat`	A Lightroom Classic catalog, opened read-only with the standard SQLite driver	Settings text, digest, virtual-copy name, history step names
`capture-one`	Capture One `.cos` settings files (default `FOLDER/CaptureOne/Settings*`)	Complete XML

```
uv run python develop_settings.py capture ARCHIVE lr xmp FOLDER
uv run python develop_settings.py capture ARCHIVE lrc lrcat FOLDER \
  --catalog CATALOG.lrcat
uv run python develop_settings.py capture ARCHIVE c1 capture-one FOLDER
uv run python develop_settings.py compare ARCHIVE lr lrc
uv run python develop_settings.py compare ARCHIVE lr c1 \
  --map XMP-crs:Exposure2012=CaptureOne:AL:Exposure
```

Each capture writes `ARCHIVE/settings/LABEL.jsonl.gz` (one record per photo: photo name, source identity, typed `settings` and the unchanged raw source) and `LABEL.provenance.json` (schema, source, photo count, record SHA-256, tool commit). A label is never overwritten. Typed settings use one namespace: Adobe settings as `XMP-crs:<name>`, whether they came from XMP or a catalog, and Capture One settings as `CaptureOne:<block>:<name>` (for example `DL` for the main adjustments and `AL` for the block holding automatic values). Text that is exactly a number or boolean is stored as one; everything else stays text.

`settings.sqlite3` indexes every typed value; the `setting_values` view lists `label, app, photo, tag, value_json, number`. `reindex` rebuilds it from the captures after checking each record SHA-256. `compare` reports, per tag, the photos present in both captures, how many are identical (numbers by value), and for numbers the median and 90th-percentile differences and the correlation.

There are two implementations with the same command line and archive format:

	Python `develop_settings.py`	Rust `develop-settings`
Reads XMP with	ExifTool (`compare_dng.metadata`)	The XMP packet in a DNG's first IFD or a JPEG's APP1 segments, parsed directly
Records and provenance	Pydantic models	serde types
`raw` for `xmp`	ExifTool tags and warnings	The XMP packet text

Build the Rust one with `make build-rust` (or `cargo build --release --bin develop-settings` in `rust-scanner`) and run it as `rust-scanner/target/release/develop-settings capture ...`. Both type values the same way, name settings by their XMP property (ExifTool's `ColorTemperature` is recorded as `XMP-crs:Temperature`, its XMP name and Lightroom Classic's), flatten structures to `Parent/Field` and write the same canonical `value_json`, so either can add captures to an archive, reindex it or compare it.

`test_develop_settings.py` checks that both write equal records from the same fixtures and that each reindexes and compares the other's captures identically.

Settings in rendered JPEGs

Exported JPEGs are a second place to find settings, and often the only record of what an export actually used:

Rendering	Settings in the JPEG
Lightroom and Lightroom Classic exports	The complete Camera Raw settings (`crs:`), including the Auto marker when Auto was applied
Capture One exports	None; read the RAW folder's `.cos` files instead
Out-of-camera JPEGs	Only EXIF basics such as white balance mode, contrast, saturation and sharpness

```
uv run python develop_settings.py capture ARCHIVE lr-export xmp EXPORT_FOLDER \
  --extensions jpg
```

A capture records which extensions it read in its provenance. The Rust reader takes the XMP from the JPEG's APP1 segments, including Adobe's extended XMP, whose chunks it assembles by GUID and offset. Exports are a good cross-check: in one 446-photo test, the settings embedded in Lightroom's Auto exports matched the settings Lightroom had written into the DNGs on every photo, and the Lightroom Classic exports matched its catalog.

Capture One's `.cos` files hold the current state, including any manual adjustment made after Auto; compare their modification times with the export times before treating them as the settings of an older export.

Limits:

- An `xmp` capture reads the files and fails if one changes during the read. The Rust reader reads the XMP packet stored in a DNG's first IFD or a JPEG's APP1 segments.
- Values are typed exactly: the Rust build parses floats with serde_json's `float_roundtrip`, so every stored number reads back bit for bit.
- Lightroom Classic computes Auto lazily and stores `-999999` placeholders until a photo is rendered. `lrcat` refuses such photos unless `--allow-pending` is given, and then records the placeholders with a note.
- Editors use different controls and scales. `--map` pairs tags; it does not claim that the controls are equivalent.
- Captures describe settings; they do not reproduce rendered appearance and nothing is written back to photos, catalogs or settings files.

Rust and OCaml extraction pools

`metadata-extract WORKERS` reads a JSON array of UTF-8 paths from stdin. The OCaml equivalent `metadata-extract-oxcaml WORKERS` reads NUL-delimited paths. Both emit ExifTool JSON records in input order without deserializing and reserializing their metadata values in the native worker.

Both launch one ExifTool process per path with the same extraction flags and allow 1–36 workers. Rust schedules them through threads; OCaml multiplexes stdout/stderr pipes from child processes. The expensive metadata parser remains ExifTool in both cases. Rust or OCaml execution does not make its metadata a DNG pixel digest.

To select a pool for the portable snapshot:

```
DNG_METADATA_BACKEND="$PWD/rust-scanner/target/release/metadata-extract" \
DNG_METADATA_WORKERS=4 \
```

```
uv run python portable_metadata.py /media/source/foto /media/reference/foto \
  archives/source --batch-size 16
```

The OCaml route uses `DNG_METADATA_BACKEND="$PWD/metadata-extract-oxcaml"`.

Optional exiftool-rs engine

`DNG_METADATA_ENGINE=exiftool-rs` replaces the per-file Perl process with the `exiftool-rs` crate, a pure Rust reimplementation of ExifTool mirroring ExifTool 13.59 tables, running inside the native worker. The build links the crates.io release 0.12.0, pinned exactly, which carries the `-struct` support and numeric-output fixes that dng-compare contributed upstream (listed in `XMP.md`). It is off by default and needs `make build-exiftool-rs`, which rebuilds `metadata-extract` with the `exiftool-rs` cargo feature, builds the standalone OCaml project in `ocaml-exiftool-rs` with dune (OxCaml switch `5.2.0+ox` by default; set `OXCAML_SWITCH=` for the dune on `PATH`), and imports its worker as `metadata-extract-exiftool-rs-oxcaml`. The engine is GPL-3.0-or-later; binaries built with it carry that license. Run it after `make build`: a later `make build-rust` rebuilds `metadata-extract` without the feature, and that binary rejects `--engine exiftool-rs` with an explicit error.

```
DNG_METADATA_ENGINE=exiftool-rs \
DNG_METADATA_BACKEND="$PWD/metadata-extract-exiftool-rs-oxcaml" \
DNG_METADATA_WORKERS=8 \
uv run python portable_metadata.py /media/source/foto /media/reference/foto \
  archives/source --batch-size 16
```

`preserve_years.py --metadata-engine exiftool-rs --metadata-workers N` sets the same variables, records `metadata_engine` and `metadata_engine_version` in the provenance, and archives the engine sources with the tools. Workers accept `--engine exiftool|exiftool-rs` and `--engine-version`; the Perl workers reject the other engine instead of silently substituting it.

Records from this engine are a separate metadata identity. They follow the same `-j -a -G0:1:4 -n -struct -b -u -U` shape and exclusions, with `CopyN` numbering for duplicate tag names in discovery order, nested XMP structures and ExifTool's list rules, but values are still the crate's own: many composites and MakerNotes values keep their print form under `-n`, unknown MakerNotes tags under `-U` are far fewer than Perl's, and `ExifTool:ExifToolVersion` reports the crate version. Every record carries `ExifTool:DngCompareEngine`, and extraction failures are written into the record as `ExifTool:Error` so a failed file remains a failed receipt. Compare archives produced by the same engine only; the Perl flags remain the reference extraction, and `exiftool_version` in the provenance stays the Perl version.

The OCaml binding builds with stock OCaml 5 (`Domain.spawn`) and with OxCaml (`Domain.Safe.spawn` over portable closures), capping workers at the runtime's recommended domain count; the C stub releases the runtime lock so domains run the Rust engine in parallel. Parity with Perl ExifTool on real DNG archives has not been measured here; the crate's published parity claims cover its own corpus. `DNG_METADATA_PREFETCH_BATCHES` defaults to four when a native pool is selected and accepts 1–16. `DNG_METADATA_SHARDS` can run independent extraction/compression shards, with the total native worker budget apportioned across them. A separate index owner receives bounded atomic batches. Shard count must not exceed the worker budget or 16. Only the coordinator writes its checkpoint; workers do not compete for the same SQLite connection.

Year-scoped preservation workflow

`preserve_years.py` wraps byte preservation, full metadata snapshots, XMP extraction, image categorization and final verification for explicit year trees. Supply repeatable labeled source roots and one reference root:

```
uv run python preserve_years.py archives/years \
  --source-root archive-a=/media/archive-a/foto \
  --source-root archive-b=/media/archive-b/foto \
  --reference-root /media/reference/foto \
  --metadata-workers 4 --preservation-workers 2 \
  --metadata-backend rust --preservation-backend rust \
  --database-backend sqlite --metadata-shards 2
```

Only four-digit year directories under those supplied roots become tasks. This is not an automatic whole-volume intake. The source label becomes the archive namespace `LABEL/YEAR` and must remain stable on resume. The matching ledger prefers the same relative path, then a unique case-sensitive filename within the reference year; an ambiguous or missing reference causes full-original fallback. A match never bypasses the subsequent byte verification.

Option	Purpose
`--source-root LABEL=PATH`	Explicit source photo root; repeatable
`--reference-root PATH`	Root containing corresponding reference years
`--start-at LABEL/YEAR`	Skip earlier tasks only after their retained verification passes
`--rate-mib N`	Preservation pacing budget
`--metadata-workers N`	Native ExifTool budget; zero selects serial extraction
`--preservation-workers N`	Byte-preservation pool, 1–36
`--metadata-backend`: `rust`, `oxcaml`	Native extraction implementation
`--preservation-backend`: `python`, `rust`, `oxcaml`	Byte/reconstruction implementation
`--metadata-shards N`	Independent bounded extraction/compression owners
`--database-backend`: `sqlite`, `turso`	Checkpoint engine
`--sequential-pass`	Byte stage followed by metadata
`--unified-pass`	Shared inventory with overlapping byte/metadata work; default

Unified mode lets metadata batches proceed only once the corresponding byte preservation has succeeded. Each stage retains its own checkpoint and database owner. It avoids a second independent inventory in the overlapping phases; it does not avoid reading whatever payloads are needed for actual identity proof.

The workflow records source/reference facts, matching decisions, tool hashes, compiler/extraction settings and available Git state in provenance. Tools are copied into the archive for reproducibility. Keep those artifacts with the payloads. A root-level summary is only the aggregate of its explicitly scoped years, not certification of unrelated lrmobile folders, other volumes or an entire Lightroom library.

A retained archive created by the former hardcoded volume workflow has different scope metadata. Resume it with its archived producing tools or use a new generic output directory; do not silently replace its old scope with newly supplied roots. A same-looking year label does not establish the same source inventory.

Finalization and image categorization

`image_categories.py ARCHIVE` compares DNG raw and auxiliary data after byte preservation completes. Exact whole-file matches can reuse that stronger byte identity; changed copies use the Python image comparison; missing references remain `missing_reference`. `DNG_IMAGE_WORKERS` controls the 1–36 image pool and `DNG_IMAGE_RATE_MIB` controls its pacing estimate. Different encoded images can still require substantial decoding memory.

`finalize_year.py ARCHIVE` validates completed year coverage, recovery records, metadata index integrity, batch checksums, image report coverage, saved originals or delta reconstruction, and source/reference identities. It writes `verification.json`, initially marking validation unfinished before any final acceptance. Inspect its actual issues and constituent reports. Metadata warnings, image differences and recoverable exact originals are separate facts, not one boolean to infer from a log line.

`finalize_preservation.py` is a retained single-archive finalization tool. `export_verified.py` exports only eligible raw-equal Python results for specified roots into the Rust TSV format. `sample_storage_ratio.py` samples archive storage ratios; it is an estimate over that archive, not a predicted universal saving.

```
uv run python finalize_preservation.py archives/source
uv run python export_verified.py reports/full/cache.sqlite3 reports/python-equal.tsv \
  --left-root /media/source/foto --right-root /media/reference/foto
uv run python sample_storage_ratio.py archives/source --output archives/storage-pilot
```

The single-archive finalizer expects the completed byte/metadata/XMP archive layout, not an arbitrary comparison report. The year finalizer additionally expects the year workflow's provenance and tools layout. Both set `safe_to_delete_source` false; validation is archive evidence, not automatic deletion approval. Finalizers take their archive as a positional argument; they are not argparse `--help` interfaces. The export and sampling tools do provide `--help`. Export refuses to overwrite its output. There is no source photo deletion command in this release.

SQLite and optional Turso

SQLite is the ordinary checkpoint backend. Turso support is pinned to embedded 0.8.3 and uses explicit MVCC transactions during work. The filename extension `.sqlite3` alone does not identify which engine owns a checkpoint; retain its `.engine.json` marker. The wrapper can export a closed checkpoint for legacy reading, but an active archive is not a generic file-copy snapshot.

Migration refuses an uncheckpointed SQLite WAL and keeps a `before-turso` copy. Do not remove ownership markers, edit an active database with another engine, or copy only its main file while the writer has WAL/MVCC state. A retained PID is a liveness guard; inspect a failed owner rather than clearing its state blindly. Ordinary comparison databases are project checkpoints, not Lightroom catalogs.

The Rust `turso-probe` and OCaml `turso_probe_oxcaml.ml`/C bindings are experimental smoke tests for typed values and concurrent writers. The OCaml FFI build requires its matching Turso SDK library and is not part of the ordinary build. Do not confuse a probe's 64 inserted synthetic rows with a photograph comparison or production archive acceptance. `make build-turso` reports the separate supported build path; read its output before using the experiment.

Measure workers without disturbing a real job

Worker count is a measurement, not a core-count rule. HDD seeks, compressed metadata output, ExifTool startup, decoded arrays, database ownership and cache warmth can dominate different phases. Measure a representative, disposable archive scope and compare full extracted values, not just throughput or counts.

The retained benchmark scripts cover native metadata timing, metadata memory, byte verification and the Turso metadata workflow. They write measured inputs, backend choices, typed-value checks and tool hashes to reports. They are research tools with explicit archive/year/filter arguments, not automatic tuning services. Run their `--help` before selecting a scope.

```
uv run python benchmark_metadata_workers.py archives/years --year archive-a/2023
uv run python benchmark_metadata_memory.py archives/years
uv run python benchmark_byte_workers.py archives/years --year archive-a/2023
uv run python benchmark_turso_metadata.py archives/years --year archive-a/2023 \
  --path-contains /selected-album/ --dataset leica
```

The timing benchmark requires 18 selected DNG source/reference pairs and tests worker counts 1, 2, 4, 8, 12, 18, 24 and 36. `--path-contains` explicitly narrows relative paths. The memory benchmark uses the prior timing report's exact input paths. The Turso `--dataset` value labels the result; it does not verify a camera model or select a private camera tree. These scripts still sample live source bytes and create archive-side research outputs, so only run them on a scope you intend to read.

The optional live-job benchmark mode uses SIGSTOP/SIGCONT on a declared owned process tree to isolate a trial. Only opt into that after understanding the selected PID and its descendants. A stopped process can retain locks or an unfinished transaction; stopping an arbitrary Lightroom, database or copy process is unsafe. A Python `finally` attempts resumption during ordinary cleanup, but SIGKILL, a crash or machine shutdown can prevent it. The public benchmark mode requires an explicit pause option and validates the process's ownership and preservation command. Prefer an idle machine and no live PID.

RSS reports sample processes and ExifTool descendants; summed RSS can count shared pages more than once and does not equal unique physical memory. A warmed short metadata trial is not cold multi-terabyte HDD throughput. Report worker count, files, bytes, codec/layout strata, wall time, cache state and whether the measurement includes extraction, compression, indexing and validation.

Recovery after a failure

First determine whether a process is still alive and whether its outputs show actual progress or a terminal error. A stale status file or a command timeout is not proof that the job died. Never start another archive owner because a progress line has been quiet.

Retain the failed report, logs, checkpoint and source version. Rerun the same qualified owner only after its prior process is gone and the source/reference scope still matches. Do not turn an exception into a fabricated equal result. For a decoder failure, use a supported codec/layout or retain the exact original. For a source identity change, determine whether software wrote metadata and start a new qualified comparison rather than weakening the guard.

Temporary `.partial`, `.tmp` or `.recovery-partial` files are incomplete unless their normal owner finished verification and commit. A payload in an archive folder is not sufficient evidence that the whole archive completed. Check the manifest, checkpoint, final summary and restore. For reference-backed storage, check both delta/original payload and the required reference version.

Before deleting any source tree through a separate process, account for all regular files and sidecars, renamed candidates, unique metadata/settings, filesystem attributes, catalog locations and the required independent copies. Perform a verified restore on representative changed and missing-reference cases. A true raw subsumption verdict is useful evidence, but is only one part of that decision.

Boundaries of this release

- Comparison discovery is DNG-specific and name-based; it is not a universal RAW-to-DNG conversion matcher or an automatic renamed-photo detector.
- Selected TIFF page order and tags define the stored-sample scope. Unsupported layouts and codec failures stay unverified.
- Equal stored samples do not mean equal renderings, edits or cloud records.
- No Lightroom catalog, native digest registration, Sync queue or cloud payload is modified.
- ExifTool-visible fields do not cover every private/unparsed byte. Whole-file recovery covers payload bytes when metadata reports cannot explain them.
- Rust and Python caches are filesystem-identity reuse, not an always-fresh cryptographic reread; their identity policies differ deliberately.
- Some low-level tools depend on exclusive ownership by their caller. In particular, Rust report directories are not automatically locked, and Rust metadata batches need an unchanged input pair set and source scope.
- Byte archives can depend on immutable reference copies. Full-copy mode avoids that dependency at the cost of storage.
- A complete report describes the declared inventory and direction. It does not certify unrelated folders, public cloud convergence or deletion eligibility.

The useful result is an explainable archive decision: what matches, what differs, what remains unverified, and exactly how a retained original can be reconstructed.